

Momentum MCP

Interview & Recruiter Brief

A local-first continuity layer for AI assistants

30-second version

I built Momentum MCP after noticing that my real productivity problem was not creating tasks - it was reconstructing context. I wanted a simple way for an AI assistant to know what I was working on, what I was waiting for, what decisions had been made, and why. Momentum stores that context locally and exposes it through narrow MCP tools, so the reasoning layer can change without losing the user's state.

GitHub: github.com/pratham-sharma-10/Projects/tree/main/momentum-mcp

Why I built this

I did not start with "I want to build an MCP server." I started with a pattern I kept seeing in my own work: I could create tasks easily, but I kept losing the context around them. A follow-up might live in one note, a decision in another conversation, and the reason behind a project choice somewhere else. When I returned to the work later, I had to rebuild the story in my head.

That led to a simple product question: **what if an AI assistant could remember where I left off without needing my entire life uploaded to another cloud service?**

The problem in one line

People do not only lose tasks. They lose context between tasks, conversations, decisions, projects, and follow-ups.

What problem does Momentum solve?

- Task managers tell me what to do, but usually not why that task exists.
- AI chats can be useful in the moment, but I did not want the product to depend on one assistant's conversation history.
- Notes preserve information, but they do not naturally distinguish between an open commitment, something I am waiting on, and a decision I already made.
- When I switch between projects, I often spend time reconstructing "where was I?" before I can actually continue working.

Momentum turns those loose pieces into a small set of durable objects: **projects, people, context, open loops, and decisions.**

Main use cases

Use case	What it helps with
Daily brief	See open loops, priorities, overdue items, and anything I am waiting on.
Waiting on someone	Track an external dependency separately from a normal task.
Project handoff to myself	Ask "where did I leave this?" and recover recent context, open loops, and decisions.
Decision memory	Store not only what was decided, but why it was decided.
Relationship follow-up	Keep lightweight context around a person and an unresolved follow-up.
Search	Find related tasks, notes, or decisions without remembering where I stored them.

How anyone can use it

There are two ways to use Momentum.

1. Local CLI - no AI client required

- momentum brief - see a daily summary.
- momentum add "Follow up Friday" --project "Job Search" --priority high - create an open loop.
- momentum waiting - see external dependencies.
- momentum search "Amazon" - find related context.
- momentum resolve 3 - close an item.

2. MCP client - for conversational use

An MCP-compatible AI client can call the Momentum tools directly. A user can ask "What am I waiting on?" or "Where did I leave this project?" and the AI retrieves structured state from Momentum instead of guessing from chat history.

Important privacy choice

The real database lives outside the GitHub repository at ~/.momentum-mcp/momentum.db. The public repo contains source code, tests, docs, and fictional demo data only.

Tech stack

Technology	Why it is there
TypeScript	Main language for the server, CLI, tests, and tooling.
Node.js 24	Runtime, including built-in SQLite support.
Model Context Protocol (MCP)	Lets compatible AI clients discover and call Momentum tools.
SQLite	Local persistence for projects, people, context, decisions, and open loops.
Zod	Runtime validation for MCP tool inputs.
GitHub Actions	Runs build, tests, and the privacy check in CI.
Git + GitHub	Version control, public documentation, and portfolio distribution.

Architecture, in plain English

AI client or local CLI -> Momentum service -> SQLite database

The AI is responsible for reasoning. Momentum is responsible for trustworthy state, retrieval, and bounded actions.

I intentionally separated reasoning from state. That means I do not need to embed a specific model API into the project. A user can keep the same Momentum data while changing the AI client they prefer.

Why MCP instead of just building another app?

A normal productivity app could solve part of this problem, but MCP made the product more interesting for three reasons:

- 1. Portability:** the same structured context can be used by different MCP-compatible AI clients instead of being locked into one chatbot.
- 2. Tool boundaries:** I can expose narrow actions such as "get waiting items" or "record decision" rather than handing an AI unrestricted access to a whole database.
- 3. Separation of concerns:** Momentum owns state; the AI client owns reasoning. That keeps the core product simpler and more model-agnostic.

Why not just use a task manager? Because the core unit is not only a task. Momentum also stores dependencies, project context, people, and decision rationale.

Why not just rely on AI memory? I wanted the important state to be explicit, inspectable, portable, and under the user's control rather than tied to one assistant's hidden memory or chat history.

Privacy and guardrails

- Local-first storage by default.
- Private database is outside the Git repository.
- The app refuses to initialize private storage inside a Git worktree unless a deliberate development override is used.
- No telemetry in the core project.
- No model API key is required by Momentum itself.
- Debug logging is metadata-only.
- Public tests and examples use fictional data.
- A privacy-check script scans the repository for common secret patterns and private database/key files.

Why this matters to me as a PMM

The biggest learning was not "how to call an MCP tool." It was learning how product choices become positioning choices.

- **Context:** What information does an AI actually need to be useful?
- **Permissions:** What should an agent be allowed to read or change?
- **Trust:** How do privacy and user control affect adoption?
- **Value proposition:** Is the product solving a workflow problem, or only adding "AI-powered" language?
- **Messaging:** How do I explain an agentic product in concrete user outcomes instead of technical jargon?

For me, the value proposition became much clearer than "an MCP server for productivity":

"Your AI remembers where you left off."

How I would explain it in an interview

60-second answer

"I built Momentum MCP after noticing that my biggest productivity cost was not creating tasks, but rebuilding context when I returned to work. I wanted a local-first layer that could store open loops, project context, people, and decision rationale, and make that state available to an AI through MCP. I built it in TypeScript on Node with SQLite, added a local CLI so it is useful even without an AI client, and designed privacy into the architecture by keeping real data outside the repo and using narrow tools. What made the project interesting for me as a PMM was seeing how technical decisions around context, permissions, and privacy directly shape positioning, trust, and adoption."

If a recruiter asks "what did you personally do?"

- Defined the product problem and the core user questions.
- Reduced the data model to a small set of primitives instead of building a full task manager.
- Chose MCP because I wanted the state layer to work underneath different AI clients.
- Designed the local-first privacy model and repository guardrails.
- Built and tested the server, CLI, persistence layer, and documentation.
- Dogfooded the product locally and used fictional public demo data for sharing.

What I would improve next

- Better ranking for what should appear in the daily brief.
- Deduplication and conflict handling for repeated context.
- Optional calendar/email integrations with explicit permission boundaries.
- Encrypted backups and an export/import format.
- A lightweight visual interface for non-terminal users.
- More user testing to validate whether "continuity" is the right framing and which workflows create the most value.

Recruiter quick facts

Project	Momentum MCP
One-line description	Local-first continuity layer that lets AI assistants retrieve open loops, project context, people, and decision rationale.
Built with	TypeScript, Node.js, MCP, SQLite, Zod, GitHub Actions

Interfaces	MCP server + local CLI
Privacy model	Real data stays local and outside the public repository by default.
Public repo	github.com/pratham-sharma-10/Projects/tree/main/momentum-mcp
Best demo	momentum brief; momentum waiting; momentum context --project "Momentum MCP"
What it demonstrates	Product thinking, technical fluency, MCP/tool design, local-first architecture, privacy guardrails, testing, and PMM framing.